

AlgoBOWL: Enclose Horse

The puzzle game *Enclose Horse* is played on a two-dimensional grid filled with water, grass, apples, bees, cherries, portals, and a singular horse. The objective of the game is to place walls to prevent the horse from reaching the perimeter of the grid while maximizing the enclosed area the horse can graze.

A web version of this game can be found here:

<https://enclose.horse>

You are encouraged to play a few rounds of this game to gain an understanding of the rules.

Your Task

Implement a solver for *Enclose Horse*. You are to be given inputs and are to create the best solution you can while maximizing your score.

Scoring

The score of a given horse enclosure is computed as the total score of all tiles the horse can reach.

- Empty grass tiles, the grass tile with the horse, and grass tiles with portals all score +1 point
- A grass tile with an apple scores +11 points (+1 for the grass tile, and +10 for the apple)
- A grass tile with bees scores -4 points (+1 for the grass tile, and -5 for the bees)
- A grass tile with cherries scores +4 points (+1 for the grass tile, and +3 for the cherries)

Valid Enclosure

A valid enclosure must satisfy the following conditions:

- There is no path the horse can take from its present location to the perimeter using grass tiles and portals.
- No wall has been placed on a tile containing water, an apple, bees, cherries, or a portal.

Input Format

The first line of the input contains a single non-negative integer W the wall budget.

The second line of the input file contains two space-separated, positive integers, R and C , indicating the number of rows and columns in the grid respectively.

The next R lines each contain exactly C characters. Each character must be one of the following symbols:

- # — Water tile
- . — Empty grass tile
- H — Grass tile containing the horse
- W — Grass tile containing a pre placed wall¹
- a — Grass tile with an apple
- b — Grass tile with bees
- c — Grass tile with cherries
- p — Grass tile with a portal

¹Due to the inability of placing walls on apples, bees, cherries, and portals, it is nontrivial to find a solution to the *Enclose Horse* problem. Thus, in your input you will be required to provide a valid configuration of walls which encloses the horse.

The next line contains a non-negative integer P indicating the number of pairs of portals in the grid above (there should be exactly $2P$ characters in the grid with the symbol p).

The next P lines of input each contain four space-separated, non-negative integers, indicating the locations of the paired portals. The i th line should contain integers $r_{i,1}$, $c_{i,1}$, $r_{i,2}$, and $c_{i,2}$ — the 0-indexed row and column of the first portal and second portal respectively.

An example input is shown in Listing 1.

```
5
9 13
#####..#
#...#...#
.WHW..a.#.p.#
#.W.#...#...#
#...#####
#...#.....#
#.p.#.b.#.c.#
#...#...#...#
#.#####...###
1
2 10 6 2
```

Listing 1: Example input to the *Enclose Horse* problem

Input Restrictions

- $3 \leq R, C \leq 10^4$
- $R \times C \leq 10^4$
- Number of portals in the grid is equal to $2P$
- Number of preplaced walls in the grid is less than or equal to W
- The horse cannot reach the perimeter tiles

Output Format

The first line of output should contain a single integer — the total score of the tiles enclosed.

The next R lines each contain exactly C characters — the grid of tiles matching the input with walls enclosing the horse. The number of walls in the solution must be less than or equal to W . The only tiles which may differ from the input are preplaced walls replaced with grass tiles and grass tiles now replaced with walls.

An example output for the input shown in Listing 1 is shown in Listing 2

```
50
#####WW#
#...#...#
W.H...a.#.p.#
#...#...#
#...#####
#...#.....#
#.p.#.b.#.c.#
#...#...#...#
#W#####...###
```

Listing 2: Example output

A solution does not need to be optimal to be valid, and many valid solutions may exist for any given input.